

August AHT: the list and the working

All 60 prompt-only taskers who booked August time or led a task into audit, ranked fastest to slowest on median AHT, and the complete method behind the numbers — every query, every filter, every judgement call, and the checks that say it is right.

MEAN AHT	MEDIAN AHT	TASKS	CONTRIBUTORS
1.98 _h	1.52 _h	907	60
\$323 per task	\$248 per task	905 with claim data	49 credited · 11 with no output

01 The list

Every contributor whose sheets role is exactly `prompt_tasker` and who booked August 2026 time or was credited with a task reaching `audit` in August. **Sorted fastest to slowest on median AHT.** The eleven who booked hours but were never credited with a task have no AHT to sort on, so they sit at the bottom. Hours are measured work per task; median is the number to use.

60 contributors · 905 tasks · 1,789.1 August payable hours · \$291,936. Pooled mean 1.98h, median 1.52h. Shaded rows have 10+ tasks and are the only ones worth ranking on. *Median \$ is that person's median hours at their own contract rate, not the \$163.17 blended rate used for the cohort-wide \$323 / \$248 figures.*

#	CONTRIBUTOR	\$/H	TASKS	MEDIAN H	MEAN H	MIN H	P90 H	MAX H	MEDIAN \$	AUG PAYABLE H	AUG COST \$
1	Mitchell Kimball	175	1	0.15	0.15	0.15	0.15	0.15	26	2.9	515
2	Mia Chamoun	150	2	0.16	0.16	0.16	0.16	0.16	24	2.2	328
3	Alema Louis	150	2	0.20	0.20	0.13	0.13	0.26	29	8.0	1,204
4	Alex Fey	200	30	0.32	0.93	0.00	2.38	4.29	63	46.9	9,383
5	Daurys gonell	150	1	0.53	0.53	0.53	0.53	0.53	80	0.0	0
6	Daniel Zalevsky	150	21	0.61	0.94	0.08	1.50	5.06	91	46.1	6,909
7	Hinda Diakite	150	8	0.75	1.88	0.01	4.34	5.92	112	18.5	2,779
8	Nadine Maglia	175	2	0.94	0.94	0.34	0.34	1.54	164	3.2	553
9	Alexander Koblansky	150	23	1.14	1.43	0.01	1.66	8.70	171	47.6	7,138

#	CONTRIBUTOR	\$/H	TASKS	MEDIAN H	MEAN H	MIN H	P90 H	MAX H	MEDIAN \$	AUG PAYABLE H	AUG COST \$
10	Pravit Kochar	150	16	1.16	1.18	0.06	2.83	2.97	173	26.8	4,012
11	Travis Crevasse	150	1	1.42	1.42	1.42	1.42	1.42	212	5.0	750
12	Frank Collier	150	58	1.44	2.02	0.01	4.01	9.54	216	84.3	12,647
13	Elliot Saye	200	77	1.48	1.48	0.03	2.79	4.70	296	100.1	20,020
14	Ariya Savla	150	9	1.48	1.39	0.08	2.42	3.10	223	13.2	1,973
15	Johnny Tran	150	16	1.49	1.46	0.05	2.83	3.55	223	66.6	9,995
16	Ran Meshulam	150	31	1.49	2.21	0.12	3.67	7.55	223	76.2	11,425
17	Jack Sorensen	150	3	1.50	1.61	0.16	1.50	3.17	224	5.0	757
18	Joy Tokosi	150	25	1.50	2.33	0.03	4.43	7.23	225	104.2	15,637
19	David Hua	200	42	1.50	1.65	0.00	2.73	6.09	300	40.2	8,037
20	Charles Mack	150	27	1.51	2.03	0.07	4.33	7.33	226	55.7	8,349
21	Alex Moon	150	35	1.51	1.40	0.01	2.92	4.38	227	101.0	15,153
22	Marc Levy	150	8	1.53	1.57	0.05	2.71	2.78	230	13.5	2,028
23	Siddhant Kumbhat	150	44	1.57	1.58	0.11	3.03	4.46	236	72.3	10,839
24	Diego Inaki Pardo de Orellana	175	14	1.58	1.52	0.18	2.66	3.44	277	29.3	5,135
25	Sutton Zhang	150	95	1.61	2.11	0.00	3.85	6.73	241	88.1	13,222
26	Artus Granier	150	26	1.69	2.20	0.04	3.45	9.91	253	38.2	5,728
27	Spencer Dumas	150	26	1.71	2.66	0.02	4.96	10.54	256	20.8	3,115
28	John Wang	175	68	1.72	1.97	0.03	4.12	7.50	301	110.1	19,275
29	Francisco Cerqueira	175	5	1.80	2.27	0.14	3.63	4.33	315	11.3	1,983
30	Will Briley	150	13	1.90	2.29	0.12	3.73	6.23	285	24.1	3,607
31	Caroline Rawe	150	3	1.96	1.36	0.04	1.96	2.08	293	3.2	483
32	Tyler Collingbourne	150	17	2.02	2.48	0.02	2.91	10.90	303	33.0	4,950
33	Jared Minor	200	13	2.07	2.10	0.58	2.56	4.34	414	127.5	25,510
34	Paul Dykes	150	2	2.12	2.12	1.42	1.42	2.83	318	2.9	440
35	Ethan Ohana	150	34	2.17	2.50	0.05	4.34	10.28	325	49.9	7,485
36	Kevin Cao	150	3	2.25	3.37	2.19	2.25	5.66	338	17.5	2,622
37	Mikel Pardo	150	4	2.31	2.78	0.87	3.09	5.63	346	25.8	3,867
38	Melvan Hoo Jun Hong	200	42	2.33	2.63	0.01	4.00	12.53	465	71.9	14,374
39	Saiyid Kazmi	150	1	2.39	2.39	2.39	2.39	2.39	359	4.3	647
40	Jack Colyar	150	4	2.48	2.48	0.06	2.96	4.91	373	26.3	3,947
41	Jonathan Smith	150	2	2.63	2.63	1.84	1.84	3.43	395	8.2	1,230
42	Liam Tuohy	150	25	2.86	3.40	0.12	6.88	10.28	429	41.5	6,233
43	Andy Deng	150	10	3.19	2.78	0.24	4.97	5.10	478	9.1	1,371
44	Shahwaiz Faisal	150	3	3.48	2.70	0.10	3.48	4.51	523	6.8	1,024
45	Diogo Pires	175	3	3.61	4.18	3.30	3.61	5.63	632	10.6	1,851
46	Ali Mallick	150	4	3.63	3.60	3.31	3.66	3.84	544	19.8	2,974
47	Albert Park	150	1	3.97	3.97	3.97	3.97	3.97	595	6.0	906
48	Shreyas Rajkumar	150	4	4.43	5.04	1.42	7.44	9.91	664	10.7	1,612
49	Winston Curteza	150	1	8.12	8.12	8.12	8.12	8.12	1,218	9.3	1,396
50	Eshan Tapuria	175	— no task reached audit in August · 2 active days							1.3	225
51	Jacob Coran	175	— no task reached audit in August · 1 active day							0.5	79
52	Vamshi Eppanapally	150	— no task reached audit in August · 2 active days							8.5	1,274

#	CONTRIBUTOR	\$/H	TASKS	MEDIAN H	MEAN H	MIN H	P90 H	MAX H	MEDIAN \$	AUG PAYABLE H	AUG COST \$
53	James Hayakawa	150	—	no task reached audit in August · 5 active days						3.3	491
54	Luis Rubano	150	—	no task reached audit in August · 1 active day						1.0	146
55	Tim Marino	150	—	no task reached audit in August · 5 active days						8.2	1,224
56	Julius Schenk	175	—	no task reached audit in August · 2 active days						0.3	62
57	Troye Bullock	150	—	no task reached audit in August · 10 active days						4.4	659
58	Davit Hakobyan	150	—	no task reached audit in August · 4 active days						6.0	908
59	Yvan Nathanael Ngongang	150	—	no task reached audit in August · 5 active days						8.4	1,255
60	Jared Curran	150	—	no task reached audit in August · 3 active days						1.3	193

How to read the sort

The order is honest but the top and bottom of it are not informative. Both ends are dominated by people with one to four credited tasks, where a single task decides the whole ranking: **Mitchell Kimball at 0.15h and Winston Curteza at 8.12h each have exactly one task**. A median over one number is that number.

Twenty-five people cleared 10 tasks. Inside that group the spread is **0.32h (Alex Fey, 30 tasks) to 3.19h (Andy Deng, 10 tasks)** — still roughly **10×** inside a single job title, and that is the real finding. The people worth acting on first are the ones who are both slow and high-volume, because they carry the most hours: **Melvan Hoo Jun Hong** (2.33h median × 42 tasks, \$200/h), **Liam Tuohy** (2.86h × 25), **Ethan Ohana** (2.17h × 34) and **Tyler Collingbourne** (2.02h × 17). Between them that is about 118 tasks running roughly an hour over cohort median, which is on the order of **\$18k–\$20k a month** of avoidable time.

At the other end, **Alex Fey** (0.32h over 30 tasks) and **Daniel Zalevsky** (0.61h over 21) are fast enough that they are worth checking rather than celebrating — verify their audit pass rate before holding them up as the target, since this metric measures time to reach audit, not quality once there.

The eleven with no credited task

Rows 50–60 booked **43.1 payable hours and \$6,517** in August — **2.4%** of the cohort's August cost — without a single task reaching audit. Most logged one to five active days at small hours, which reads as onboarding or ramp rather than leakage. **Troye Bullock** is the one row that looks different: 10 active days spread across the month for 4.4 hours and still nothing credited, which is worth a direct question rather than an assumption.

Reconciliation: 59 people had August payable hours, 48 of them were credited with a task, 11 were not. One further person (Daurys gonell) has a credited task but no August payable time — $48 + 11 + 1 = 60$ rows.

02 The work, step by step

Everything below was run read-only against `tsip_prd` through the Metabase dataset API. Each step gives the query, the result, and any judgement call it forced.

STEP 1 · DEFINE THE POPULATION

Who counts as a “prompt-only tasker”?

First establish whether anyone holds more than one role, since the brief was to exclude them.

```
SELECT m.role::text, count(*) AS rows_, count(DISTINCT m.user_id) AS users
FROM tsip_project_members m
WHERE m.project_id = 'sheets'
GROUP BY m.role;

-- and, to prove nobody holds two roles:
SELECT c, count(*) FROM (
  SELECT user_id, count(*) AS c FROM tsip_project_members
  WHERE project_id='sheets' GROUP BY 1
) x GROUP BY c;
```

Result. 618 sheets members. Every one has exactly **one** role row — the “rows per user” query returns a single bucket, `1 → 618`. So multiple roles do not occur in this data and nobody was excluded on that basis.

Roles: `beginner_tasker` 369 · **`prompt_tasker` 112** · `calibration_tasker` 90 · `reviewer` 19 · `decomp_prompt_tasker` 10 · `calibration_failed` 7 · `decomp_prompt_reviewer` 3 · `decomp_tasker` 3 · `beginner_tasker_reviewer` 3 · `eval_hallucination_reviewer` 2.

Judgement call: the filter is `role = 'prompt_tasker'` exactly. This drops the hybrids (`decomp_prompt_tasker`, `decomp_prompt_reviewer`) as instructed — and also drops people doing prompt work under a reviewer or beginner title. Most consequentially it excludes the project's highest-volume contributor, who is `beginner_tasker_reviewer`. The role field is a permissions label, not a description of work.

STEP 2 · DEFINE THE TASK SET

Unique tasks that reached audit in August

“Unique” is enforced by taking each task's **first** entry into `audit` — tasks can re-enter, and counting entries instead of tasks would double-count. Day bucketing is `America/New_York` via the double convert, per sheets ground truth (DB timestamps are naive UTC; a single convert shifts the day boundary).

Result. 1,287 unique tasks first entered `audit` between 2026-08-01 and 2026-08-31 ET, across all roles.

STEP 3 · THE CREDIT RULE

Who does a task belong to?

The obvious reading — whoever triggered the transition into `audit` — turns out to be wrong. I checked it before relying on it:

```
WITH tp AS (  
  SELECT t.id AS task_id FROM tsip_tasks t  
  JOIN tsip_project_versions pv ON pv.id = t.project_version_id  
  WHERE pv.project_id = 'sheets')  
SELECT DISTINCT ON (tst.task_id)  
  tst.task_id, tst.created_at, tst.triggered_by, tst.from_state, tst.event  
FROM tsip_state_transitions tst JOIN tp ON tp.task_id = tst.task_id  
WHERE tst.to_state = 'audit' AND tst.from_state IS DISTINCT FROM 'audit'  
ORDER BY tst.task_id, tst.created_at;  
-- then group the August rows by the trigger's role
```

Result — August audit entries by trigger role.

(no membership row)	via	<code>review_gate → APPROVE</code>	1,248	(97.0%)
<code>eval_hallucination_reviewer</code>	via	<code>SEND_FORWARD_TO_AUDIT</code>	29	
<code>beginner_tasker_reviewer</code>	via	<code>APPROVE</code>	4	
<code>reviewer</code>	via	<code>ADMIN_OVERRIDE</code>	3	
<code>decomp_prompt_reviewer</code>	via	<code>APPROVE</code>	1	
<code>prompt_tasker</code>	2	(both <code>ADMIN_OVERRIDE</code>)		

97% of tasks are pushed into audit by a system account. Taking the trigger literally would return two tasks for the entire month. So credit follows the sheets ground-truth convention instead: **the last person to move the task out of `edit_task` or `redo_task` before it was approved** — whoever's work led it there.

```

cred AS (
  SELECT fa.task_id, fa.audit_at,
    (SELECT e.triggered_by
      FROM tsip_state_transitions e
      WHERE e.task_id = fa.task_id
            AND e.from_state IN ('edit_task','redo_task')
            AND e.from_state IS DISTINCT FROM e.to_state
            AND e.created_at <= fa.audit_at
      ORDER BY e.created_at DESC
      LIMIT 1) AS user_id
  FROM fa)
SELECT c.task_id, c.user_id
FROM cred c
JOIN tsip_project_members m
  ON m.user_id = c.user_id AND m.project_id = 'sheets'
WHERE m.role::text = 'prompt_tasker';

```

Result. Of 1,287 August tasks, **907** are credited to a contributor whose role is exactly `prompt_tasker`. The remaining 380 belong to the other nine roles and are out of scope.

STEP 4 · MEASURE TIME ON THE TASK

Time on that task specifically, not allocated

There is no per-task time field in the platform — payable time is project-grain. The only per-task signal is `tsip_task_claims`: each row is one person holding one task, with `claimed_at` and `last_activity_at`. I sum the credited person's claims on that exact task, capping each claim at 12 hours so one forgotten claim cannot dominate.

```

SELECT pt.task_id, pt.user_id,
  sum(LEAST(GREATEST(EXTRACT(EPOCH FROM (
    COALESCE(c.last_activity_at, c.closed_at, c.claimed_at) - c.claimed_at
  ))/3600.0, 0), 12) )
  FILTER (WHERE c.user_id = pt.user_id) AS own_h,
  sum(LEAST(GREATEST(EXTRACT(EPOCH FROM (
    COALESCE(c.last_activity_at, c.closed_at, c.claimed_at) - c.claimed_at
  ))/3600.0, 0), 12) ) AS all_h,
  count(c.id) FILTER (WHERE c.user_id = pt.user_id) AS claims_own
FROM pt LEFT JOIN tsip_task_claims c ON c.task_id = pt.task_id
GROUP BY 1, 2;

```

Result — raw claim-hold hours. 907 tasks, of which **905** have a claim record for the credited person (2 do not and are dropped).
 mean **16.76** claim-hours · median **12.89** · total **15,171**

counting every contributor's claims rather than just the credited one: mean 21.55, median 17.09

These raw numbers are not work hours. A claim is wall-clock possession — people keep a task claimed overnight and over weekends. 16.76 hours per task is obviously not what anyone worked. Step 5 fixes this.

STEP 5 · CALIBRATE

Convert possession time into paid work time

Pull what these same people were actually paid for in August — a completely independent source that never touches the claims table.

```
-- independent of claims entirely: what these people were actually paid for
WITH pt AS (SELECT user_id FROM tsip_project_members
             WHERE project_id='sheets' AND role::text='prompt_tasker'),
rev AS (SELECT DISTINCT ON (ptr.payable_time_entry_id)
        ptr.id, ptr.total_seconds, ptr.started_at, pte.user_id
        FROM payable_time_revisions ptr
        JOIN payable_time_entries pte ON pte.id = ptr.payable_time_entry_id
        WHERE ptr.project_id='sheets' AND ptr.started_at >= '2026-07-28'
        ORDER BY ptr.payable_time_entry_id, ptr.revision_number DESC)
SELECT sum(s.total_seconds)/3600.0 AS aug_payable_hours
FROM rev r
JOIN payable_time_revision_segments s ON s.revision_id = r.id
WHERE (s.started_at AT TIME ZONE 'UTC' AT TIME ZONE 'America/New_York')::date
      BETWEEN '2026-08-01' AND '2026-08-31'
      AND r.user_id IN (SELECT user_id FROM pt)
      AND r.user_id NOT IN (SELECT id FROM internal);
```

Result. 59 `prompt_tasker` accounts booked **1,789.1 payable hours** in August, costing **\$291,936** — a blended **\$163.17/hour**.

The calibration is then one division:

```
calibration factor = 1,789.1 payable hours / 15,171 claim-hours
                  = 0.11793

mean  16.76 claim-h x 0.11793 = 1.98 work hours -> $323/task
median 12.89 claim-h x 0.11793 = 1.52 work hours -> $248/task
```

Note what this does: it makes the **mean exact by construction** — 1,789.1 hours ÷ 907 tasks = 1.97h, which is the mean with no modelling in it at all. The claims data is only doing one job:

distributing that known total across tasks so a *median* and a distribution can exist.

STEP 6 · CHECK IT

Does the answer survive scrutiny?

- **Independent reconciliation.** The top-down figure (payable hours ÷ tasks = 1.97h) and the bottom-up scaled mean (1.98h) agree. They share the numerator by construction, so this checks the *task count*, not the hours.
- **Against the wider pipeline.** An earlier, separate calculation over Jul–Aug and all roles gave a blended AHT of 2.46h. August prompt-only at 1.98h is lower, which is the right direction: that figure included decomp, calibration and beginner time.
- **Cap sensitivity.** With no per-claim cap, raw mean rises to 50.5 claim-hours and median to 22.6 — the calibration absorbs the mean shift entirely, but the **median would move**. This is the single largest uncertainty in the median and is stated rather than hidden.
- **Coverage.** 905 of 907 tasks (99.8%) have claim data. The 2 without are dropped rather than counted as zero, which would have dragged the median down.
- **Row limits.** The Metabase dataset API truncates at 2,000 rows. Every distribution statistic here is computed server-side with `percentile_cont`, so nothing is silently cut. This bit me earlier in the project and is worth knowing.

03 What could still be wrong

- **Claim-hold is a proxy for effort, not a measurement of it.** Someone who claims a task, works 20 minutes, and releases it 6 hours later looks identical to someone who worked 6 hours. Across 905 tasks this should average out; for any single task it may be badly wrong. **Do not use a per-task figure to judge a person.**
- **The 12-hour cap is a judgement, not a measurement.** It is the main lever on the median.
- **Credit goes to the last submitter before audit.** A task reassigned late gives all the credit to whoever finished it, even if someone else did most of the work. 92% of send-backs are self-fixed, so this is usually the same person — but not always.
- **The role filter excludes real prompt work** done under reviewer or beginner titles, including the project's biggest producer. This is the requested scope, not an accident.

- **Everything is blended across task varieties.** Patch-based still cannot be separated from greenfield in platform data.
- **One inconsistency worth noting:** Daurys gonell is credited with an August task but has no August payable time, so their hours come from a claim that spans the month boundary. Immaterial at one task, but it is the kind of edge that shows the two data sources are not perfectly aligned.

Source: `tsip_prd` via Metabase (database 2, read-only), queried 31 Aug 2026. Tables:

`tsip_project_members`, `tsip_state_transitions`, `tsip_tasks`, `tsip_project_versions`,
`tsip_task_claims`, `payable_time_entries` / `_revisions` / `_revision_segments`,
`contributor_rates`, `tsip_users`. Conventions from `projects/sheets/project.md` (task credit,
ET double convert, audit-as-complete). Queries and a 905-row per-task CSV are on `claude/aht-`
`cost-estimation-sql-u969zu` under `aht/`.